GPU Memory Snapshot Implementation Handoff

Mizan.ai — Modal serving layer · Target: QwenBrain (A10) and QwenLite (T4) · modal_app.py / model_loader.py · Prepared July 24, 2026

Reference: Modal's official vLLM + GPU memory snapshot pattern (the **lfm_snapshot.py** example, and the Mistral 3 case study showing cold starts drop from roughly 118s to roughly 12s). This is an established pattern with real reference code, not something to build from first principles.

Goal

Reduce cold start latency for QwenBrain and QwenLite by snapshotting GPU memory state (loaded model weights, compiled kernels, CUDA graphs) after model load, so future cold starts restore from a snapshot instead of re-running the full vLLM engine initialization.

Status: this is an experimental Modal feature as of July 2026. Test thoroughly before treating it as production-reliable. Known limitations: multi-GPU snapshotting is unreliable, and any GPU graphics/CUDA activity prior to the snapshot point can cause failures.

1. App / Class Decorator Changes

For both **QwenBrain** and **QwenLite** `@app.cls(...)` decorators, add the two flags below. These are additive — no existing args (gpu, volumes, secrets, scaledown_window, startup_timeout, timeout, max_containers) need to change.

```
enable_memory_snapshot=True
experimental_options={"enable_gpu_snapshot": True}
```

2. Split the Enter Hook Into Snap and Post-Snap Phases

Currently both classes have a single `@modal.enter() def load(self):` that runs `ensure_model_on_volume(...)` then `load_vllm_engine(...)`. This needs to become two methods:

```
@modal.enter(snap=True)
def load_and_snapshot(self):
```

Runs **before** the snapshot is taken. Download/locate weights, construct the vLLM engine, and warm it up with a few dummy forward passes (per Modal's recommendation) so CUDA graph capture and JIT compilation happen here, not on the first real request. **Important:** put the vLLM engine to sleep (see Section 3) before this method returns, so the KV cache is empty/discarded prior to the snapshot.

```
@modal.enter(snap=False)
def wake_up(self):
```

Runs **after** the snapshot is restored, on every cold start (including the very first one, immediately following `load_and_snapshot`). Wake the vLLM engine back up from sleep mode here — this recreates the KV cache fresh rather than restoring stale/meaningless cache pages from the snapshot itself.

Modal considers the container ready to receive requests once both enter methods have exited, so the request-handling code (the `generate` / `generate_lite` endpoint methods) does not need to change.

3. vLLM Sleep Mode Integration

vLLM has a native sleep mode designed for exactly this handoff. Required changes in **model_loader.py**:

- load_vllm_engine() needs to expose the LLM engine object such that sleep()/wake_up() can be called on it. Check the current vLLM version's API — this may be llm.sleep() and llm.wake_up(), or via the engine's internal executor. Confirm exact method names against vLLM 0.22.1 docs, since this API has moved around across versions.
- Before the snapshot is taken (end of load_and_snapshot): call engine.sleep() (or equivalent) to discard the KV cache. Modal's own guidance is that it's cheaper to discard the unfilled KV cache before snapshotting and rebuild it on restore than to snapshot and restore meaningless cache pages.
- After restore (start of wake_up): call engine.wake_up() (or equivalent) to reallocate a fresh KV cache before accepting requests.

Reference implementation to copy the pattern from: Modal's **lfm_snapshot.py** example (modal-labs/modal-examples repo, path `06_gpu_and_ml/llm-serving/lfm_snapshot.py`) and the Ministral 3 example at modal.com/docs/examples/ministral3_inference — both show the full sleep-mode-plus-snapshot wiring for a vLLM-served model end to end. Use these as the direct template rather than building the pattern from scratch.

4. Snapshot Cache Invalidation

Add a snapshot_key-style version string (see Modal's basic embedder example) that gets bumped whenever the model weights, quantization config, or engine kwargs change, so a stale snapshot is never restored against incompatible code. Simplest approach: tie it to the model repo id and vLLM engine kwargs already defined at the top of modal_app.py (QWEN_BRAIN_REPO, QWEN_LITE_REPO, etc.) — if those change, force a fresh snapshot by redeploying.

5. What Does Not Need to Change

- The FastAPI endpoint methods (generate, generate_lite) — unchanged.
- run_vllm_chat() — unchanged, still called the same way post-wake_up.
- Translator and Embedder classes — out of scope for this change (much lower cold start cost already; can be revisited later using the simpler pattern in Modal's basic gpu_snapshot.py example if desired, but not required now).
- GPTQ quantization config, dtype="float16", TRITON_ATTN attention backend for QwenLite — all unaffected by snapshotting.

6. Validation Checklist Before Deploy

- Confirm vLLM 0.22.1's exact sleep()/wake_up() method names and required arguments (a level parameter may exist to control how much state is discarded vs. retained).
- Test cold start end to end: deploy, let the container scale to zero, send a request, confirm the response is correct AND measure the time — compare against the current baseline cold start time for both QwenBrain and QwenLite.
- Confirm tool-calling still works correctly after a snapshot-restored cold start. This matters because tool-call parsing in run_vllm_chat is regex-based (Hermes-style <tool_call> tags) — test this together with the snapshot change, not in isolation.
- Watch Modal deploy logs for any snapshot-related warnings on first deploy.

- Since this is an experimental feature, keep a fallback path in mind: if snapshot restore ever fails or misbehaves in production, the classes should still function correctly with enable_gpu_snapshot simply removed — don't make application logic depend on snapshot succeeding.

Sources

https://modal.com/docs/guide/memory-snapshots
https://modal.com/docs/examples/gpu_snapshot
https://modal.com/blog/gpu-mem-snapshots
https://modal.com/blog/mistral-3
https://modal.com/docs/examples/lfm_snapshot
https://modal.com/docs/examples/ministral3_inference